

# SO2 - Visão geral

---

Grupo M3

---

## Integrantes

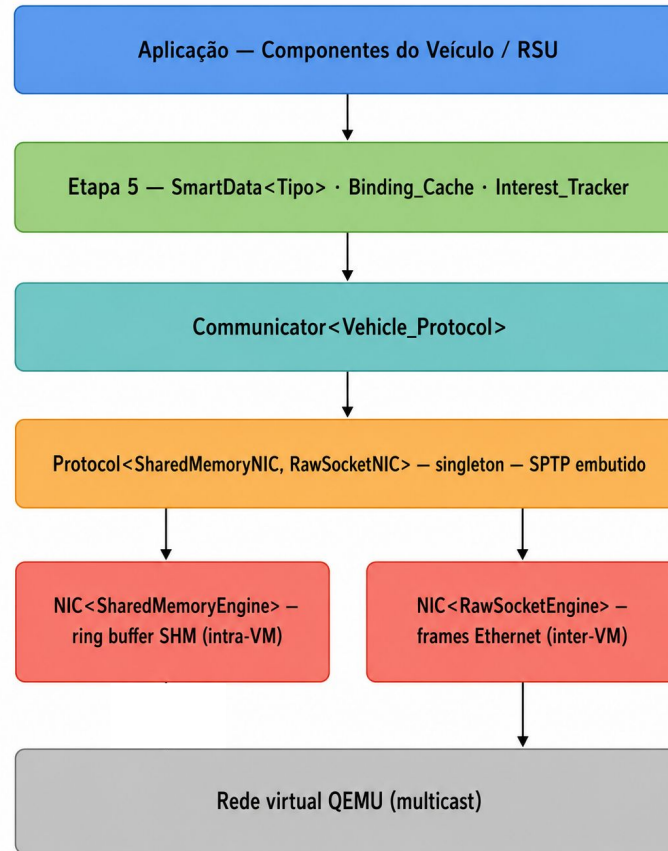
Arthur Erpen, Caetano Peruzzo e Gabriel Herrera

# ● Overview: domínio e mapeamento:

- Veículo = **VM QEMU**
- Componente (sensor, powertrain, ECU) = processo POSIX
- Presença de **RSUs**
- VMs se comunicam por uma rede virtual privada (**multicast QEMU**); dentro da VM, processos se comunicam por shared memory
- Cada etapa estende a anterior **sem reescrever**, confinando a diferença na camada certa

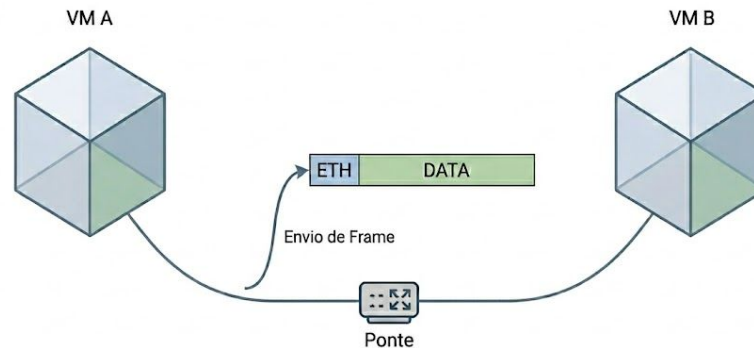
| Etapa | Foco                                               |  | Mensagem (M)                                                                                                                |
|-------|----------------------------------------------------|--|-----------------------------------------------------------------------------------------------------------------------------|
| E1    | Comunicação entre sistemas (inter-VM)              |  | $\{ . * \}$                                                                                                                 |
| E2    | Comunicação intra-sistema (entre processos)        |  | $\{ \text{address, payload} \}$                                                                                             |
| E3    | Sincronização temporal (entre quaisquer entidades) |  | $\{ \text{address, timestamp, payload} \}$                                                                                  |
| E4    | Sincronização espacial (entre quaisquer entidades) |  | $\{ \text{origin, timestamp, payload} \}$                                                                                   |
| E5    | Publish-subscribe orientado a interesse            |  | $I (\text{Interest}): \{ \text{origin, ts, type, period} \}$<br>$R (\text{Response}): \{ \text{origin, ts, type, value} \}$ |

- **Overview: arquitetura final em camadas**



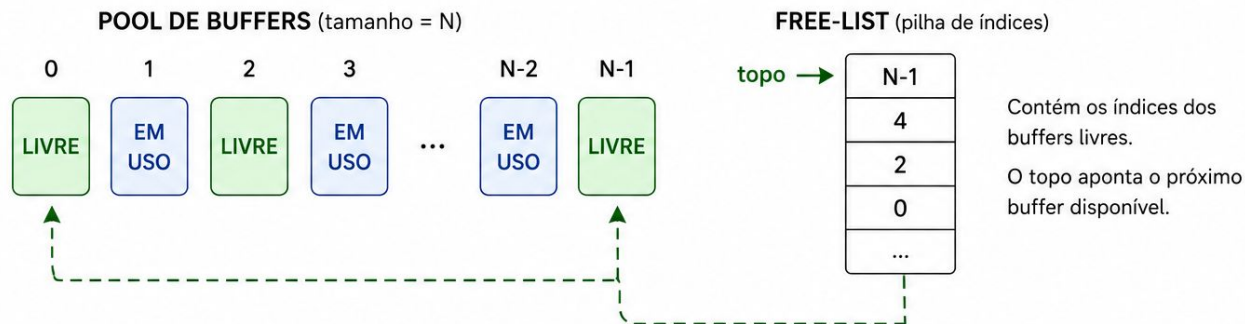
## ● E1: Engine de comunicação entre sistemas

- Objetivo (enunciado): comunicar VMs via frames Ethernet, com raw sockets, através da classe Engine da API
- Por ser a base de todo o projeto, a Etapa 1 não divergiu de forma significativa da implementação dos demais grupos.
- Como a base do código foi disponibilizada pelo professor, seguimos a mesma linha.



## ● E1 Diferencial

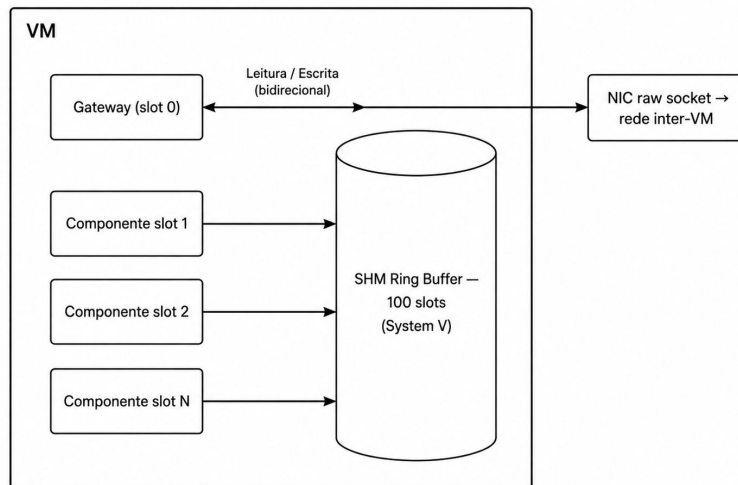
- Implementamos uma free-list nos buffers da NIC.



- Recepção por callback na Engine.
  - NIC registra callback no construtor da Engine. “Quando chegar um frame, chame esta função”.
  - Frame chega → Engine chama callback → NIC aloca buffer e notifica o Protocol.

## ● E2: Comunicação entre componentes (Engine SHM) e Diferenciais

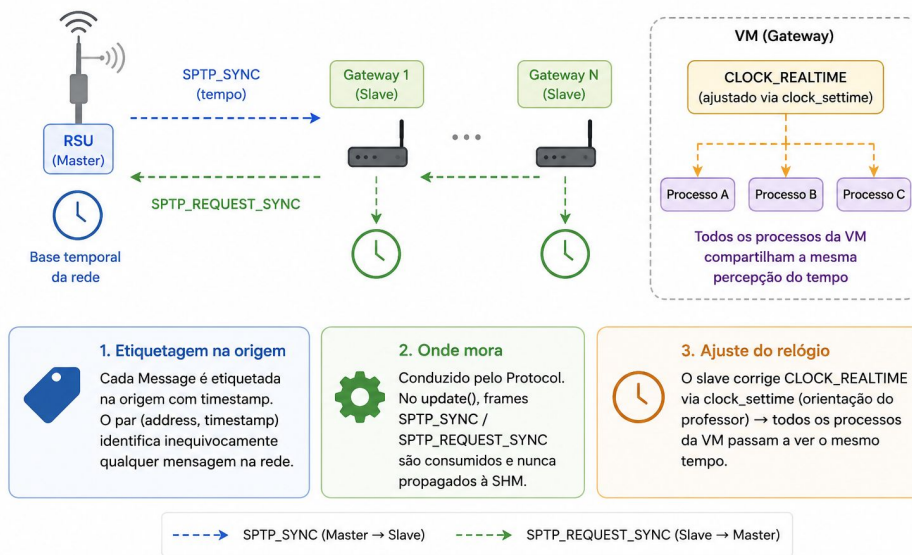
- Objetivo (enunciado): estender a API para comunicação entre componentes do mesmo veículo (IPC)
- Papéis decididos em runtime pelo slot: slot 0 = gateway, slots 1..N = componentes (sem código separado)
- Gateway como ponte: frame de componente sobe ao gateway pela SHM (flag DELIVER\_TO\_GATEWAY) e o gateway repassa à rede.
- SHM: 1 mutex do ring buffer, 1 semaphore de empty, N semaphores de full (1 pra cada consumidor)
- Saída por timeout tratada, a recepção raw usa sigtimedwait com timeout de 1s: em EAGAIN/EINTR reavalia `_running` → encerramento limpo, sem travar no sinal
- Self-drop sem MAC: na SHM o leitor ignora o próprio frame por `writer_slot == _slot`; no raw socket, por `frame.src() == meu MAC`.



## ● E3: Sincronização temporal (SPTP) + Diferenciais

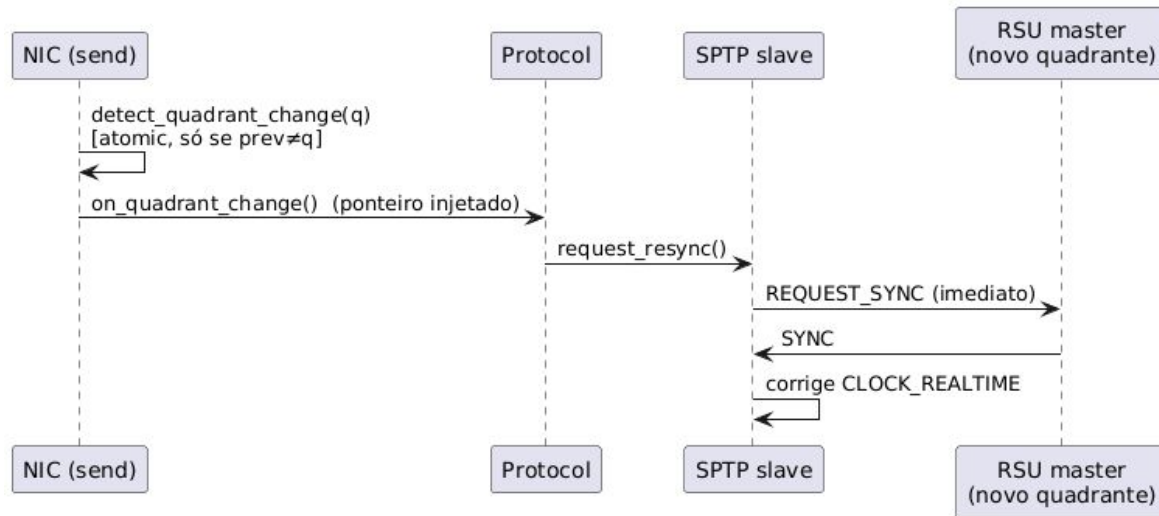
**Objetivo (enunciado):** etiquetar Message com timestamp na origem, (address, timestamp) deve identificar inequivocamente qualquer mensagem na rede, processos da mesma VM compartilham a percepção do tempo

- PTP simplificado (SPTP): RSU é a base temporal (**master**). Gateways são **slaves**.
- Onde mora: conduzido pelo **Protocol**. No update(), frame SPTP\_SYNC/SPTP\_REQUEST\_SYNC é consumido e nunca propagado à SHM.
- Slave-Driven: O slave envia REQUEST\_SYNC no seu ritmo; o master só responde SYNC → master puramente reativo, sem estado por slave nem agendador distribuído.



## ● E3 Diferenciais

- Resync **event-driven** na troca de quadrante:
  - NIC::send() detecta a troca via `_last_quadrant.exchange(q)` → só dispara em `prev ≠ q`. Chama `on_quadrant_change()` (ponteiro injetado pelo Protocol) → `_sntp->request_resync()` → `REQUEST_SYNC` imediato





## ● E4: Sincronização espacial (módulo kernel GPS) + Diferenciais

Objetivo (enunciado): etiquetar Message com coordenadas, posicionar VMs em quadrantes, sincronização espacial via módulo de kernel com interface que retorna a posição.

- GPS\_IOC\_SET\_FIXED: congela o quadrante, RSUs são fixas
- Em enable\_sync(true), o Protocol chama set\_fixed(true) → o GPS congela o quadrante da RSU  
O mesmo flag is\_master define papel temporal (master SPTP) e posição (fixa)
- NIC não conhece Protocol nem GPS — recebe 3 ponteiros para funções: read\_quadrant, write\_quadrant, quadrant\_change\_handler (funcionalidade cross-layer, a informação do quadrante está no header do protocolo, mas a NIC tem acesso)

### Interface ioctl



## ● E4 Diferencial

- Adjacências modeladas (sem diagonal):
  - Modela de forma realista a cobertura: atravessar o mapa exige passar por quadrantes intermediários



### 1) Sem timer/thread

O sorteio acontece dentro do `ioctl`,  
comparando `jiffies` com  
`last_change + 3*HZ`

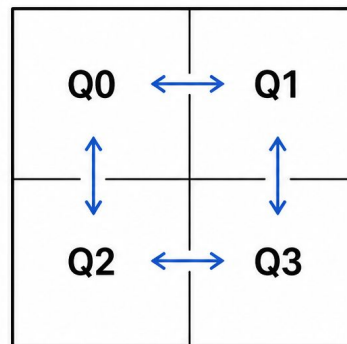


### 2) Apenas quadrantes adjacentes

Tabela de adjacência:

`gps_neighbors[4][2]`

| Quadrante | Vizinhos (adjacentes) |
|-----------|-----------------------|
| Q0        | Q1, Q2                |
| Q1        | Q0, Q3                |
| Q2        | Q0, Q3                |
| Q3        | Q1, Q2                |



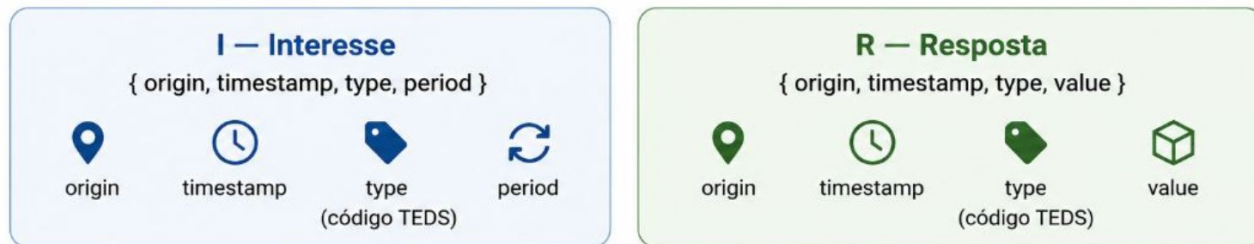
Q0 pode ir para Q1 ou Q2.

Nunca para Q3.

## ● E5: Modelo Interesse / Resposta

**Objetivo (enunciado):** Modelo publish-subscribe, todos recebem o publish (Broadcast), somente os interessados processam, interesse de acordo com tipo de dados (TEDS – IEEE1451.4)

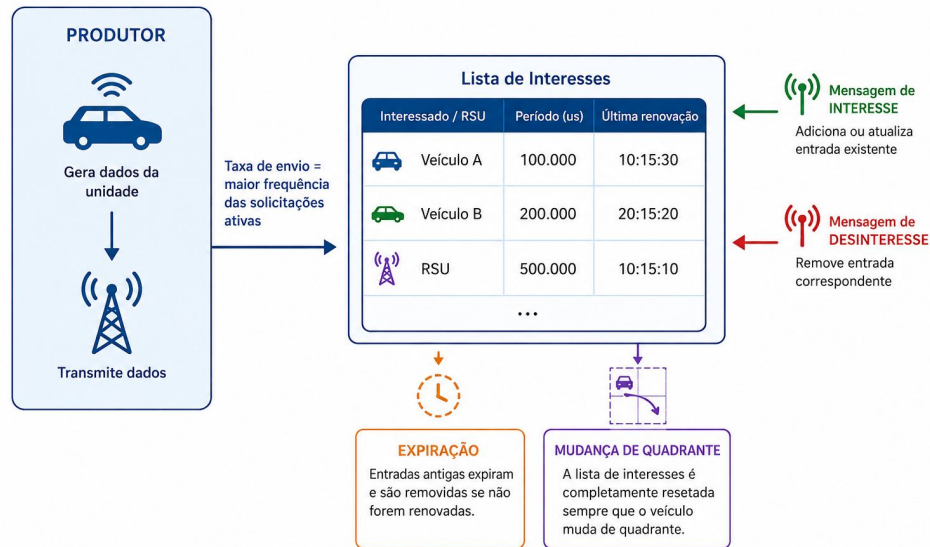
- As mensagens agora são tipadas, de acordo com o conteúdo do payload
- SmartHeader, carrega kind (INTEREST ou RESPONSE) e Unit (numero identificador do tipo de dado)
- InterestMessage: define um payload de mensagem de interesse, carrega um SmartHeader, o período em micro segundos, e um byte que identifica as mensagens como interesse ou desinteresse.
- ResponseMessage: define um payload genérico de mensagem de resposta, carrega um SmartHeader e um Valor de tipo genérico (passado via template), que é o dado em si.



## ● E5: Diferencial

**Produtor:** componente que pode gerar e transmitir dados de uma unidade específica. Ele gerencia uma lista dinâmica de Interessados, cuja taxa de envio é ditada pela maior frequência registrada entre as solicitações ativas. O ciclo de vida dessa lista possui algumas regras:

- Sempre que o produtor recebe uma mensagem de interesse ele pode adicionar uma nova entrada a lista de interesse, ou atualizar as informações de uma entrada já existente.
- Se uma mensagem de desinteresse é recebida e essa colide com alguma entrada de interesse da lista essa é removida.
- Entradas antigas expiram e são removidas se não forem renovadas pelo Interessado/RSU.
- A lista de interesses é completamente resetada sempre que o veículo muda de quadrante.



## ● E5: Diferencial

**Consumidor/Interessado:** o componente que manifesta a necessidade de receber um tipo de dado específico. Seu comportamento baseia-se em:

- **Declaração Inicial:** Declara seu interesse uma única vez assim que é inicializado no sistema.
- **Redeclaração Preventiva (Keep-alive):** Caso passe um período prolongado sem receber nenhuma atualização do dado esperado, ele retransmite a mensagem de interesse para reativar o fluxo (troca de quadrantes implicitamente).

① **Declaração Inicial**  
Declara seu interesse uma única vez ao iniciar.

② **Redeclaração Preventiva (Keep-alive)**  
Se não receber atualizações por um período prolongado, retransmite o interesse para reativar o fluxo.



- **E5: Diferencial**

**RSU:** atua como o gerenciador de tráfego de dados e presença do quadrante através de duas frentes:

- **Retransmissão de Interesses:** Escuta passivamente as mensagens de interesse geradas no quadrante e as retransmite periodicamente para garantir o alcance entre as VMs.
- **Monitoramento e Expulsão (PTP):** Monitora a presença dos veículos localmente via troca de mensagens PTP. Se um veículo cessar o envio dessas mensagens de sincronização, a RSU assume que ele deixou o quadrante. Como consequência, ela remove os interesses desse veículo de sua lista de retransmissão e propaga mensagens de **desinteresse** para cessar os envios dos produtores locais.

**OBRIGADO !!!**